

Digital Signal Processing

TUTORIAL 8



BIRMINGHAM CITY
University

WINDOWED SINC FIR AND BIQUAD IIR FILTER DESIGN

Windowed-sinc FIR Filter design and Biquad IIR filter design

1. Learning Outcomes

- Understand the concept of normalised frequency
- Understand the theory of Windowed-Sinc FIR Filter Design
- Understand IIR filter structure and coefficients.
- Understand Biquad IIR filter structure
- Be able to design IIR filter based on biquad Analogue method.
- Be able to evaluate the filter performance.

2. General Filter Formula

General form of FIR filter with order M

$$y[n] = \sum_{k=0}^M b_k x[n - k]$$

Equation 1

General form of IIR filter. The filter order is max(M,N).

$$y[n] = \sum_{l=0}^N a_l y[n - l] + \sum_{k=0}^M b_k x[n - k]$$

Equation 2

For FIR filter, the design goal is to find the coefficients “b” to meet specifications. Where “b” is the impulse response, or filter kernel which can then be applied to the signal with the convolution process. For IIR filter, the design goal is to find the coefficients of both “b” and “a” to meet specifications, instead of convolution, the filter is applied through a process called recursion.

3. Windowed-Sinc FIR filter Design

Normalised Frequency

In DSP practice, when designing a digital filter, the cut-off frequency parameter for all basic filter design functions is normalized by the Nyquist frequency. When frequency is normalised, the Nyquist frequency ($F_s/2$) is given the value of 1, and then all other frequencies are scaled accordingly. E.g. For a system with a sampling frequency at 1000Hz, the Nyquist frequency is 500Hz and therefore the frequency 300 Hz can be normalised as $300/500=0.6$.

To convert normalized frequency to angular frequency around the unit circle, multiply by π . To convert normalized frequency back to hertz, multiply by half of the sample frequency.

Sinc Function

The ideal frequency response of low pass filter looks like the following diagram shown in Figure 1:

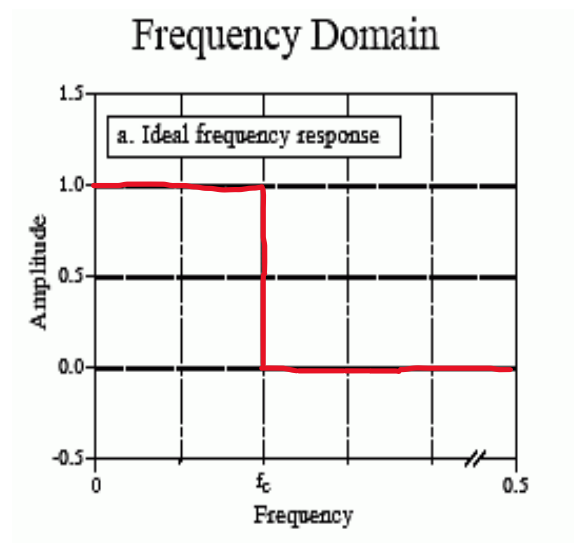


Figure 1. Ideal Frequency Response

Consider this ideal, or “brick wall”, digital low-pass filter with a cut-off frequency of ω_0 rad/s. This filter has magnitude 1 at all frequencies less than ω_0 , and magnitude 0 at frequencies between ω_0 and $F_s/2$.

The impulse response that describes a brick wall filter is called the sinc function and its sequence can be defined with the following equation:

$$\frac{1}{2\pi} \int_{-\pi}^{\pi} H(\omega) e^{j\omega n} d\omega = \frac{1}{2\pi} \int_{-\omega_0}^{\omega_0} e^{j\omega n} d\omega = \frac{\omega_0}{\pi} \text{sinc}\left(\frac{\omega_0}{\pi} n\right)$$

Equation 3

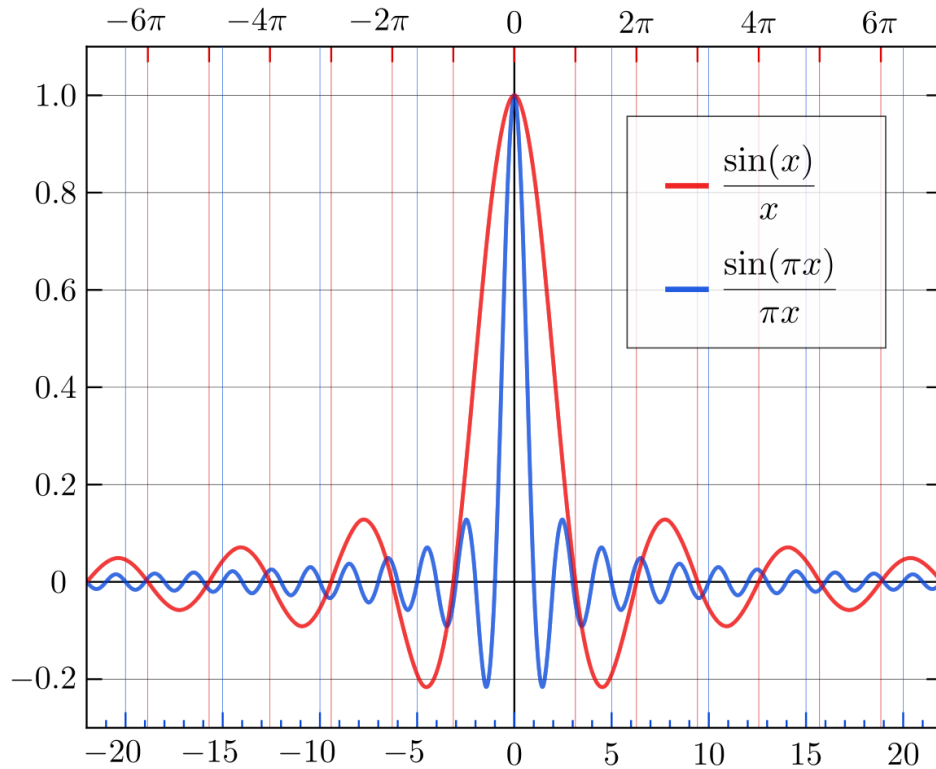


Figure 2. Sinc function in time domain

As we have previously discussed, brick wall filters are an ideal filter response, but they do not exist in the digital world. This filter is not implementable since its impulse response is infinite and non-causal. To create a finite-duration impulse response, truncate it by applying a window. Retain the central section of impulse response in the truncation to obtain a linear phase FIR filter. For example, a length 51 filter with a lowpass cutoff frequency ω_0 of 0.4π rad/s is:

$$b = 0.4 * \text{sinc}(0.4 * (-25:25))$$

Equation 3

The window applied here is a simple rectangular window. sinc function is defined as the following:

$$\text{sinc}(t) = \begin{cases} 1, & t = 0 \\ \frac{\sin(\pi t)}{\pi t}, & t \neq 0 \end{cases}$$

Equation 4

Matlab provides the built in “sinc()” function. To understand more about how to use “sinc” function, go to Matlab help file find “sinc”.

The variable ‘b’ calculated by Equation 2 can be imported as FIR filter coefficients of numerator. You can use “fvtool()” to view the spectrum of this filter:

fvtool(b,1)

Remember, for FIR filter, the denominator is always “1”. Therefore to design any windowed-sinc filter with cut-off frequency f_0 , we have:

$$b = f_0 * \text{sinc}(f_0 * (-n:n))$$

Equation 5 practical FIR design formula

Where “b” stores filter coefficients, f_0 is normalised cut-off frequency. The filter length is “2n+1”.

4. Windowed-Sinc FIR filter Design Exercise

According to Equation 4 practical FIR design formula, in Matlab write a truncated sinc function with 61 points and its frequency response should have cutoff frequency at 500 Hz when sampling frequency is 2000 Hz. Plot it using fvtool the frequency response of this filter should look like the following diagram:

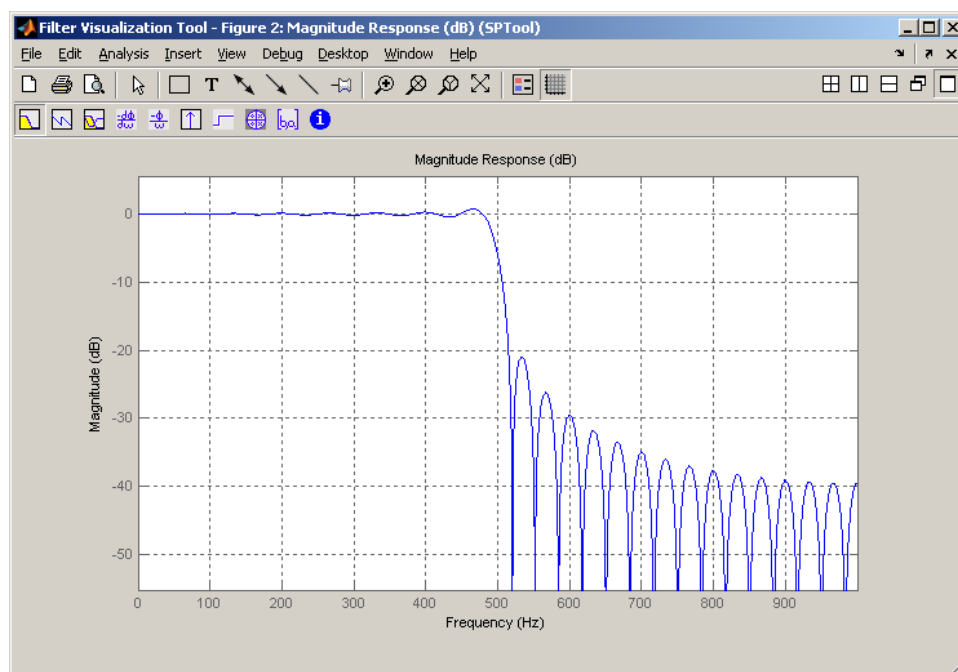


Figure 3 Sinc function designed filter

5. Biquad IIR filter Design

Unlike FIR filter, an IIR filter has both numerator coefficients: “b” and denominator coefficients: “a”. IIR filters are applied through a process called recursion instead of convolution. This involves multiplying and summing your delayed signal with the “b” coefficients in much the same way as convolution, however, the output values are then fed through a feedback loop, where they are delayed, multiplied and summed with the “a” coefficients.

The general second order IIR filter structure can be expressed as following:

$$\frac{y_n}{x_n} = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{a_0 + a_1 z^{-1} + a_2 z^{-2}} \quad \text{Equation 6}$$

Because the numerator and denominator of the transfer function are both quadratic equations, therefore this structure is called biquad digital filter.

Above Biquad IIR filter has six coefficients. It can be determined from analogue prototype by using bilinear transform. Z^{-1} indicates a delay of 1 sample, Z^{-2} is a delay of 2 samples etc.

The Biquad IIR filter can be used in parametric EQ for audio processing. Robert Bristow-Johnson wrote a handy document “Cookbook formulae for audio EQ biquad filter coefficients” regarding how to obtain the coefficients of filter from adjustable user specification. The formulas require you to provide the filter specifications and in turn the “a” and “b” coefficients are calculated

You can access this document from the following link:

<https://www.w3.org/2011/audio/audio-eq-cookbook.html> [accessible March 2023]
or at the appendix A of this lab.

To apply an IIR filter in matlab you should not use convolution instead use `filter()` with the following syntax:

```
Y=filter(b,a,X)
```

Where Y is the filtered signal. “b” are your feedforward coefficients, “a” are your feedback coefficients and X is the input signal to be filtered.

6. Tasks

- 1) Design a 61 point sinc function with cutoff frequency at 300 Hz with sampling frequency of 4000 Hz. Plot the truncated sinc function and plot the spectrum:

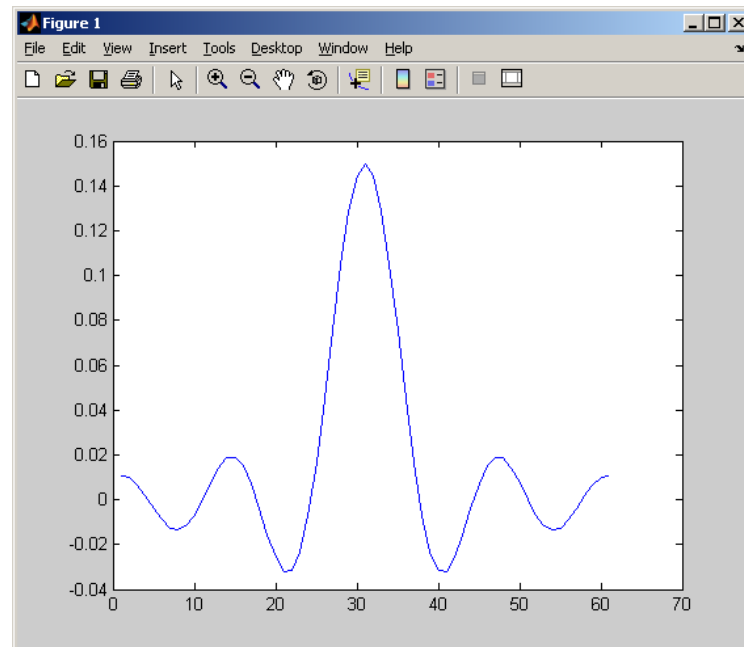


Figure 4 Truncated SINC function

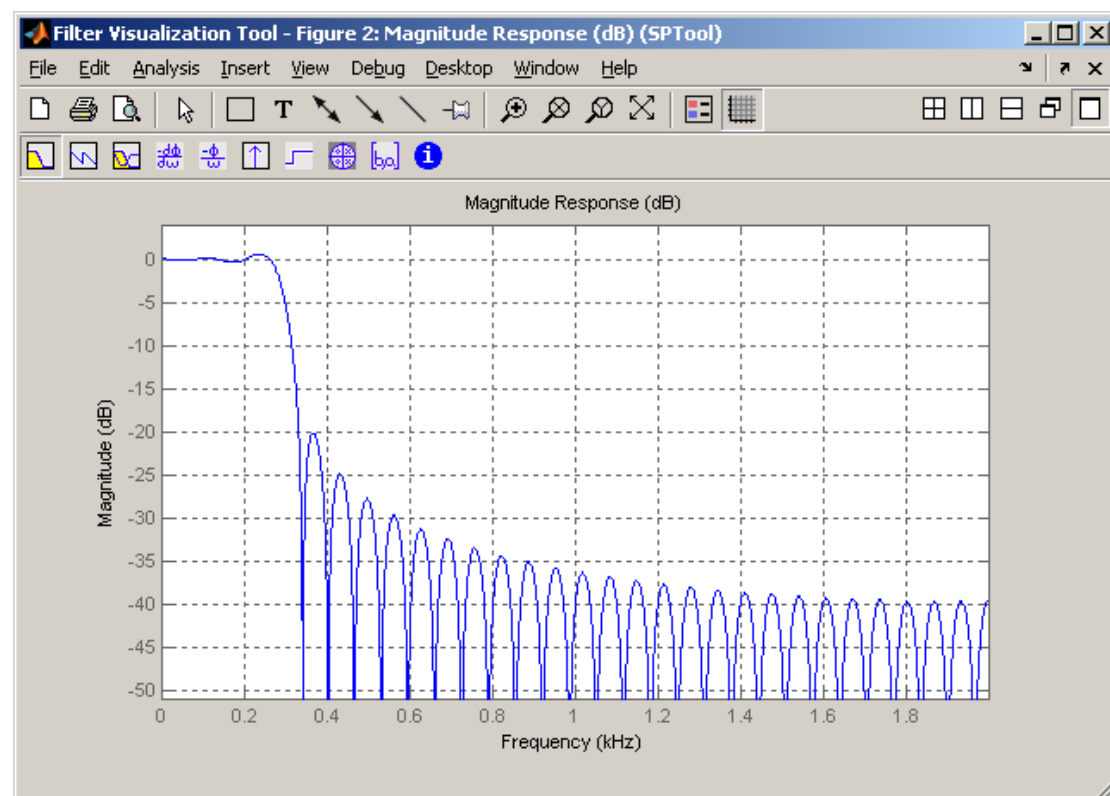


Figure 5 Frequency response of above sinc fuction

- 2) Create a signal with two frequency components 150 Hz and 800 Hz. Apply the filter generated by sinc function and compare the results.
- 3) Investigate the window functions provided by Matlab and find out how to use hamming window to improve the filter performance.
- 4) Based on Bristow-Johnson's document, design a Matlab function to implement one of the following types of filters:
 - Low-Pass filter
 - High-Pass filter
 - Band-Pass filter
 - Constant Q Band-Pass filter.

The engineering definition of “Q” factor is defined by the ratio of the bandwidth to cut-off frequency $Q = F_b/F_c$, where F_b is measured at -3dB points. F_c is central frequency if this is bandpass filter.

You can design different function structures for different types of filter. For example an audio lowpass filter function can be prototyped as:

`[b, a] = AudioEQLPF(fc, dBgain, Fs, Q)`

The function should return the IIR filter coefficients “b” and “a” for user defined filter specification with cut-off frequency being “fc”, sampling frequency “Fs” and user defined quality factor “Q”. In lowpass filter design the “dBgain” may not be used.

In other form, you can define the function to design any type of filter, for example a function prototype as below:

`[b,a] = AudioEQFilter(fc, dBgain, Fs, Q, type)`

This function is implemented in more complex approach by taking user input parameter “type”. The type parameter can be pre-defined string to represent different types of digital filter such as the table below:

Type Name	Filter Type
LPF	Low pass filter
HPF	High pass filter
BPF	Band Pass filter
BPFQ	Constant Q Bandpass filter
Notch	Notch filter
APF	All pass filter
PeakingEQ	Peaking filter
LowShelf	Low shelf filter
HighShelf	High shelf filter

After the functions are designed you can test the filter design results by using Matlab `fvtool(b,a)` function.

Appendix A

Access URL:

<https://www.w3.org/2011/audio/audio-eq-cookbook.html>

Cookbook formulae for audio EQ biquad filter coefficients

by Robert Bristow-Johnson <rbj@audioimagination.com>

All filter transfer functions were derived from analog prototypes (that are shown below for each EQ filter type) and had been digitized using the Bilinear Transform. BLT frequency warping has been taken into account for both significant frequency relocation (this is the normal "prewarping" that is necessary when using the BLT) and for bandwidth readjustment (since the bandwidth is compressed when mapped from analog to digital using the BLT).

First, given a biquad transfer function defined as:

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{a_0 + a_1 z^{-1} + a_2 z^{-2}} \quad (\text{Eq 1})$$

This shows 6 coefficients instead of 5 so, depending on your architecture, you will likely normalize a_0 to be 1 and perhaps also b_0 to 1 (and collect that into an overall gain coefficient). Then your transfer function would look like:

$$H(z) = \frac{(b_0/a_0) + (b_1/a_0) z^{-1} + (b_2/a_0) z^{-2}}{1 + (a_1/a_0) z^{-1} + (a_2/a_0) z^{-2}} \quad (\text{Eq 2})$$

or

$$H(z) = (b_0/a_0) * \frac{1 + (b_1/b_0) z^{-1} + (b_2/b_0) z^{-2}}{1 + (a_1/a_0) z^{-1} + (a_2/a_0) z^{-2}} \quad (\text{Eq 3})$$

The most straight forward implementation would be the "Direct Form 1" (Eq 2):

$$y[n] = (b_0/a_0) * x[n] + (b_1/a_0) * x[n-1] + (b_2/a_0) * x[n-2] - (a_1/a_0) * y[n-1] - (a_2/a_0) * y[n-2] \quad (\text{Eq 4})$$

This is probably both the best and the easiest method to implement in the 56K and other fixed-point or floating-point architectures with a double wide accumulator.

Begin with these user defined parameters:

F_s (the sampling frequency)

f_0 ("wherever it's happenin', man." Center Frequency or Corner Frequency, or shelf midpoint frequency, depending on which filter type. The "significant frequency".)

dBgain (used only for peaking and shelving filters)

Q (the EE kind of definition, except for peakingEQ in which $A*Q$ is the classic EE Q . That adjustment in definition was made so that a boost of N dB followed by a cut of N dB for identical Q and f_0/F_s results in a precisely flat unity gain filter or "wire".)

or BW, the bandwidth in octaves (between -3 dB frequencies for BPF and notch or between midpoint (dBgain/2) gain frequencies for peaking EQ)

or S, a "shelf slope" parameter (for shelving EQ only). When $S = 1$, the shelf slope is as steep as it can be and remain monotonically increasing or decreasing gain with frequency. The shelf slope, in dB/octave, remains proportional to S for all other values for a fixed f_0/F_s and dBgain.

Then compute a few intermediate variables:

$$A = \sqrt{10^{(\text{dBgain}/20)}} \\ = 10^{(\text{dBgain}/40)} \quad (\text{for peaking and shelving EQ filters only})$$

$$w_0 = 2\pi f_0/F_s$$

$$\cos(w_0)$$

$$\sin(w_0)$$

$$\begin{aligned} \alpha &= \sin(w_0)/(2*Q) && (\text{case: } Q) \\ &= \sin(w_0)*\sinh(\ln(2)/2 * BW * w_0/\sin(w_0)) && (\text{case: } BW) \\ &= \sin(w_0)/2 * \sqrt{(A + 1/A)*(1/S - 1) + 2} && (\text{case: } S) \end{aligned}$$

FYI: The relationship between bandwidth and Q is

$$\begin{aligned} 1/Q &= 2*\sinh(\ln(2)/2*BW*w_0/\sin(w_0)) && (\text{digital filter w BLT}) \\ \text{or } 1/Q &= 2*\sinh(\ln(2)/2*BW) && (\text{analog filter prototype}) \end{aligned}$$

The relationship between shelf slope and Q is

$$1/Q = \sqrt{(A + 1/A)*(1/S - 1) + 2}$$

$$2*\sqrt{A}*\alpha = \sin(w_0) * \sqrt{(A^2 + 1)*(1/S - 1) + 2*A}$$

is a handy intermediate variable for shelving EQ filters.

Finally, compute the coefficients for whichever filter type you want:
(The analog prototypes, $H(s)$, are shown for each filter type for normalized frequency.)

LPF: $H(s) = 1 / (s^2 + s/Q + 1)$

$$b_0 = (1 - \cos(w_0))/2$$

$$b_1 = 1 - \cos(w_0)$$

$$b_2 = (1 - \cos(w_0))/2$$

$$a_0 = 1 + \alpha$$

$$a_1 = -2*\cos(w_0)$$

$$a_2 = 1 - \alpha$$

HPF: $H(s) = s^2 / (s^2 + s/Q + 1)$

$$b_0 = (1 + \cos(w_0))/2$$

$$b_1 = -(1 + \cos(w_0))$$

$$b_2 = (1 + \cos(w_0))/2$$

$$a_0 = 1 + \alpha$$

$$a_1 = -2*\cos(w_0)$$

$$a_2 = 1 - \alpha$$

BPF: $H(s) = s / (s^2 + s/Q + 1)$ (constant skirt gain, peak gain = Q)

$$b_0 = \sin(w_0)/2 = Q*\alpha$$

$$b_1 = 0$$

$$b_2 = -\sin(w_0)/2 = -Q*\alpha$$

$$a_0 = 1 + \alpha$$

$$a_1 = -2*\cos(w_0)$$

$$a_2 = 1 - \alpha$$

BPF: $H(s) = (s/Q) / (s^2 + s/Q + 1)$ (constant 0 dB peak gain)

$$b_0 = \alpha$$

$$b_1 = 0$$

$$b_2 = -\alpha$$

$$a_0 = 1 + \alpha$$

$$a_1 = -2*\cos(w_0)$$

$$a_2 = 1 - \alpha$$

notch: $H(s) = (s^2 + 1) / (s^2 + s/Q + 1)$

$$\begin{aligned} b0 &= 1 \\ b1 &= -2*\cos(w0) \\ b2 &= 1 \\ a0 &= 1 + \alpha \\ a1 &= -2*\cos(w0) \\ a2 &= 1 - \alpha \end{aligned}$$

APF: $H(s) = (s^2 - s/Q + 1) / (s^2 + s/Q + 1)$

$$\begin{aligned} b0 &= 1 - \alpha \\ b1 &= -2*\cos(w0) \\ b2 &= 1 + \alpha \\ a0 &= 1 + \alpha \\ a1 &= -2*\cos(w0) \\ a2 &= 1 - \alpha \end{aligned}$$

peakingEQ: $H(s) = (s^2 + s*(A/Q) + 1) / (s^2 + s/(A*Q) + 1)$

$$\begin{aligned} b0 &= 1 + \alpha*A \\ b1 &= -2*\cos(w0) \\ b2 &= 1 - \alpha*A \\ a0 &= 1 + \alpha/A \\ a1 &= -2*\cos(w0) \\ a2 &= 1 - \alpha/A \end{aligned}$$

lowShelf: $H(s) = A * (s^2 + (\sqrt{A}/Q)*s + A) / (A*s^2 + (\sqrt{A}/Q)*s + 1)$

$$\begin{aligned} b0 &= A*((A+1) - (A-1)*\cos(w0) + 2*\sqrt{A}*\alpha) \\ b1 &= 2*A*((A-1) - (A+1)*\cos(w0)) \\ b2 &= A*((A+1) - (A-1)*\cos(w0) - 2*\sqrt{A}*\alpha) \\ a0 &= (A+1) + (A-1)*\cos(w0) + 2*\sqrt{A}*\alpha \\ a1 &= -2*((A-1) + (A+1)*\cos(w0)) \\ a2 &= (A+1) + (A-1)*\cos(w0) - 2*\sqrt{A}*\alpha \end{aligned}$$

highShelf: $H(s) = A * (A*s^2 + (\sqrt{A}/Q)*s + 1) / (s^2 + (\sqrt{A}/Q)*s + A)$

$$\begin{aligned} b0 &= A*((A+1) + (A-1)*\cos(w0) + 2*\sqrt{A}*\alpha) \\ b1 &= -2*A*((A-1) + (A+1)*\cos(w0)) \\ b2 &= A*((A+1) + (A-1)*\cos(w0) - 2*\sqrt{A}*\alpha) \end{aligned}$$

$$\begin{aligned}
 a0 &= (A+1) - (A-1)*\cos(w0) + 2*\sqrt{A}*\alpha \\
 a1 &= 2*((A-1) - (A+1)*\cos(w0)) \\
 a2 &= (A+1) - (A-1)*\cos(w0) - 2*\sqrt{A}*\alpha
 \end{aligned}$$

FYI: The bilinear transform (with compensation for frequency warping) substitutes:

$$(\text{normalized}) \quad s \leftarrow \frac{1}{\tan(w0/2)} * \frac{1 - z^{-1}}{1 + z^{-1}}$$

and makes use of these trig identities:

$$\begin{aligned}
 \tan(w0/2) &= \frac{\sin(w0)}{1 + \cos(w0)} & (\tan(w0/2))^2 &= \frac{1 - \cos(w0)}{1 + \cos(w0)}
 \end{aligned}$$

resulting in these substitutions:

$$1 \leftarrow \frac{1 + \cos(w0)}{1 + \cos(w0)} * \frac{1 + 2*z^{-1} + z^{-2}}{1 + 2*z^{-1} + z^{-2}}$$

$$s \leftarrow \frac{1 + \cos(w0)}{\sin(w0)} * \frac{1 - z^{-1}}{1 + z^{-1}}$$

$$= \frac{1 + \cos(w0)}{\sin(w0)} * \frac{1 - z^{-2}}{1 + 2*z^{-1} + z^{-2}}$$

$$s^2 \leftarrow \frac{1 + \cos(w0)}{1 - \cos(w0)} * \frac{1 - 2*z^{-1} + z^{-2}}{1 + 2*z^{-1} + z^{-2}}$$

The factor:

$$\frac{1 + \cos(w0)}{1 + 2*z^{-1} + z^{-2}}$$

is common to all terms in both numerator and denominator, can be factored out, and thus be left out in the substitutions above resulting in:

$$1 \leftarrow \frac{1 + 2z^{-1} + z^{-2}}{1 + \cos(w_0)}$$

$$s \leftarrow \frac{1 - z^{-2}}{\sin(w_0)}$$

$$s^2 \leftarrow \frac{1 - 2z^{-1} + z^{-2}}{1 - \cos(w_0)}$$

In addition, all terms, numerator and denominator, can be multiplied by a common $(\sin(w_0))^2$ factor, finally resulting in these substitutions:

$$1 \leftarrow (1 + 2z^{-1} + z^{-2}) * (1 - \cos(w_0))$$

$$s \leftarrow (1 - z^{-2}) * \sin(w_0)$$

$$s^2 \leftarrow (1 - 2z^{-1} + z^{-2}) * (1 + \cos(w_0))$$

$$1 + s^2 \leftarrow 2 * (1 - 2\cos(w_0)z^{-1} + z^{-2})$$

The biquad coefficient formulae above come out after a little simplification.